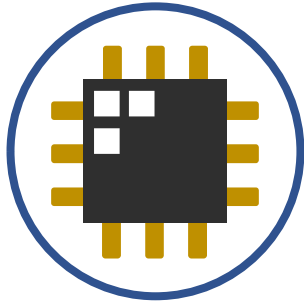
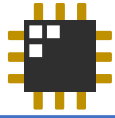




EEE 143: Switching Theory and Digital Logic Design

Lecture 1 – Number systems



How many pieces of Ferrero Rocher?

- When we count or measure things, we commonly use the **decimal** number system
- It is a *positional* number system that uses 10 as its base
- We use the following 10 symbols to represent numbers:
 - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9



8 pieces



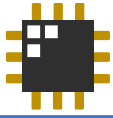
16 pieces



24 pieces

2

We developed **number systems** to express/represent values or quantities which we can use for counting (how many chocolates?), measuring (how tall are you?), or labelling things (such as the course numbers of the subjects in your curriculum). The most common number system is the **digital number system** because we most often count with the 10 fingers on our hands. However, most digital systems use a number system of a different base (the binary number system), so let us re-visit first how number systems work to understand how numbers can be represented in different ways using different bases.



Decimal numbers

- We represent our numbers with 10 symbols or digits

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

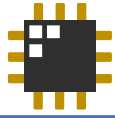
- But counting does not end in 9...
- We would need more than 1 symbol to represent larger quantities

143

$$(1 \times 10^2) + (4 \times 10^1) + (3 \times 10^0)$$

3

Decimal numbers, also known as base-10 numbers, are a common way of representing numbers in our daily life. This system uses 10 symbols or digits, ranging from 0 to 9. However, the counting does not end at 9. To represent larger numbers, we use more than one symbol or digit. For instance, the number 143 is a three-digit decimal number. It can be broken down into $1 \times 10^2 + 4 \times 10^1 + 3 \times 10^0$. Here, the digit 1 is in the hundreds place (10^2), the next digit 4 is in the tens place (10^1), and the digit 3 is in the ones place (10^0). This demonstrates how we can use multiple digits to represent larger numbers in the decimal system.



Decimal numbers

143

$$(1 \times 10^2) + (4 \times 10^1) + (3 \times 10^0)$$

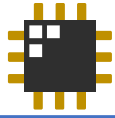
- Each digit position has different weighted values
- For decimal numbers, the digit on the left is 10 times higher than the digit on the right.

Decimal (from Latin *decimus*, meaning tenth)

4

In the decimal number system, each digit position carries a weight that is 10 times that of the position to its right. This is because the decimal system is a base-10 system. The term "decimal" comes from the Latin word "decimus", which means "tenth". This reflects the fact that each step to the left in a decimal number represents a tenfold increase in value.

For example, in the number 143, the digit 1 in the hundreds place (10^2) is 10 times the value of the digit 4 in the tens place (10^1), and 100 times the value of the digit 3 in the ones place (10^0). This pattern continues for numbers of any size, and is a fundamental characteristic of the decimal system.



Decimal numbers

- We have 10 fingers, so 10 comes out naturally!

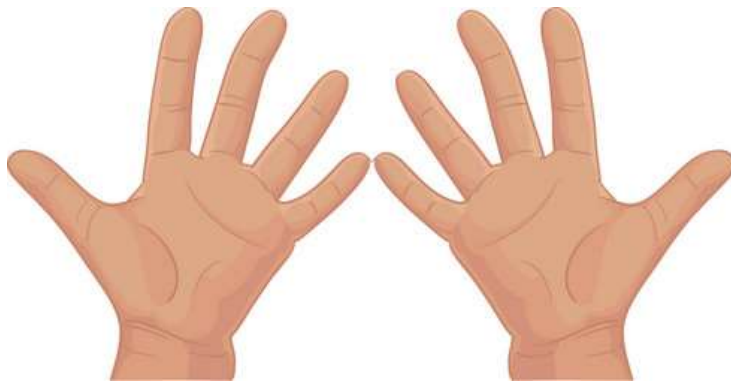
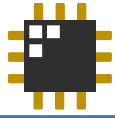


Image credit: vectorstock.com

5

The number 10 comes up very naturally because we have 10 fingers! That's how decimal numbers originally came about.



Other number systems

143

$$(1 \times 10^2) + (4 \times 10^1) + (3 \times 10^0)$$

- We have 10 digits, every digit position is a power of 10...

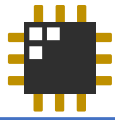
- What if we have less?



Image generated by Windows Paint Cocreator

6

So now, if we are using base 10 because we had 10 fingers, what if we had less? We'll most likely be using a different numbering system on a different base that is not 10.



Binary numbers

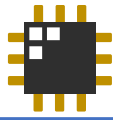
- We can represent any number in any base n , that has n digits with positional powers of n
- If we only have 2 digits, 0 and 1, how do we represent larger numbers?

$$\begin{aligned} & \mathbf{1110}_2 \quad (\text{we say 1110 in base 2, or 1110 in binary}) \\ & (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \\ & \quad \quad \quad \text{2 digits (0,1) and positional powers of 2} \\ & 1 \times 8 \quad + \quad 1 \times 4 \quad + \quad 1 \times 2 \quad + \quad 0 \times 1 \\ & \quad \quad \quad = \mathbf{14}_{10} \quad (\text{we say 14 in base 10, or 14 in decimal}) \end{aligned}$$

Indeed, just like the decimal system, we can represent any number in any base- n system, where n is the number of digits available. We can represent any number in any base n , that has n digits with positional powers of n . In the binary system, we only have two digits: 0 and 1. Therefore, we represent larger numbers using powers of 2.

For example, the binary number 1110 can be broken down as $1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$. This is equivalent to $8 + 4 + 2 + 0 = 14$ in decimal.

In the binary system, each digit position carries a weight that is 2 times that of the position to its right. This is because the binary system is a base-2 system. The digit on the left is always 2 times higher than the digit on the right. This weighted positional system allows us to represent large numbers efficiently, even with just two digits. It's the fundamental principle behind digital computers and binary logic.



Binary numbers

- Base 2 is also called binary, we call the binary digits as bits



Computers utilize binary
to manipulate data!

4-bit binary	Decimal value
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7

4-bit binary	Decimal value
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

and so on...

- If we need larger numbers, we need more bits!

8

Image credit: vectorstock.com

This counting system is in base 2, also called binary. Each symbol or digit, which can be either 0 or 1, are called bits.

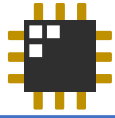
Here's a table showing the corresponding decimal values of a 4-bit binary string. We can perform the math described earlier to actually prove that each of these 4-bit binary strings are equivalent to their decimal counterparts.

When we start counting in binary, we begin with 0000, which is equivalent to 0 in decimal. Then we move to 0001, which is equivalent to 1 in decimal.

But here's where things get interesting. Since we only have two digits in binary, we've run out of digits after 1. So, to represent the next number (2 in decimal), we need to use another bit position. We reset the rightmost bit back to 0 and set the next bit to the left to 1, giving us 0010 in binary.

This process continues as we count higher. Each time we run out of digits, we add another bit position to the left and continue counting. This is how we can represent any number using just two digits in the binary system. This is analogous to the decimal numbering system, where when we start running out of digits in 1 position, we have to add 1 to the next digit position to the left. In binary, we only have 2 digits to work with, so we tend to use up the higher bit positions rather quickly. As you can see, by the time we reach decimal number 15, to represent that in binary, we already need 4 bit positions, nothing less.

Moreover, when they are laid out like this, you start to see some patterns. As you can see, the pattern in the 4-bit binary string changes as we increase the value from 0 to 15. The rightmost bit (also known as the least significant bit) toggles between 0 and 1 for every increment. The second bit from the right toggles after every 2 increments, the third bit toggles after every 4 increments, and the leftmost bit (also known as the most significant bit) toggles after every 8 increments. This pattern continues for larger binary strings as well. It's a fundamental characteristic of the binary number system.



Other bases

- Segue: How about base-1?



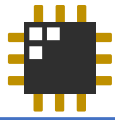
Image credit: shutterstock.com

9

A little aside and something to think about. We were focusing on binary, which is base 2, but what does it look like if we have a base 1 number system?

If you can recall from the math earlier, every digit position would be a power of the base. Thus, every bit position in base 1 number system is a power of 1. But remember, all powers of 1 are just 1 itself. Therefore, digit position has equal weights and all the weights are 1! You would need n sticks to represent the number n , for example.

Counting sticks like this, which is something you might have done as a child, is essentially using a base 1 number system.



Other bases

• Segue: How about base-60?

𐎶 1	𐎶𐎵 11	𐎶𐎶 21	𐎶𐎶𐎵 31	𐎶𐎶𐎶 41	𐎶𐎶𐎶𐎵 51
𐎶𐎶 2	𐎶𐎶𐎵 12	𐎶𐎶𐎶 22	𐎶𐎶𐎶𐎵 32	𐎶𐎶𐎶𐎶 42	𐎶𐎶𐎶𐎶𐎵 52
𐎶𐎶𐎶 3	𐎶𐎶𐎶𐎵 13	𐎶𐎶𐎶𐎶 23	𐎶𐎶𐎶𐎶𐎵 33	𐎶𐎶𐎶𐎶𐎶 43	𐎶𐎶𐎶𐎶𐎶𐎵 53
𐎶𐎶𐎶𐎵 4	𐎶𐎶𐎶𐎶 14	𐎶𐎶𐎶𐎶𐎵 24	𐎶𐎶𐎶𐎶𐎶 34	𐎶𐎶𐎶𐎶𐎶𐎵 44	𐎶𐎶𐎶𐎶𐎶𐎶 54
𐎶𐎶𐎶𐎶 5	𐎶𐎶𐎶𐎶𐎵 15	𐎶𐎶𐎶𐎶𐎶 25	𐎶𐎶𐎶𐎶𐎶𐎵 35	𐎶𐎶𐎶𐎶𐎶𐎶 45	𐎶𐎶𐎶𐎶𐎶𐎶𐎵 55
𐎶𐎶𐎶𐎶𐎵 6	𐎶𐎶𐎶𐎶𐎶 16	𐎶𐎶𐎶𐎶𐎶𐎵 26	𐎶𐎶𐎶𐎶𐎶𐎶 36	𐎶𐎶𐎶𐎶𐎶𐎶𐎵 46	𐎶𐎶𐎶𐎶𐎶𐎶𐎶 56
𐎶𐎶𐎶𐎶𐎶 7	𐎶𐎶𐎶𐎶𐎶𐎵 17	𐎶𐎶𐎶𐎶𐎶𐎶 27	𐎶𐎶𐎶𐎶𐎶𐎶𐎵 37	𐎶𐎶𐎶𐎶𐎶𐎶𐎶 47	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎵 57
𐎶𐎶𐎶𐎶𐎶𐎵 8	𐎶𐎶𐎶𐎶𐎶𐎶 18	𐎶𐎶𐎶𐎶𐎶𐎶𐎵 28	𐎶𐎶𐎶𐎶𐎶𐎶𐎶 38	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎵 48	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎶 58
𐎶𐎶𐎶𐎶𐎶𐎶 9	𐎶𐎶𐎶𐎶𐎶𐎶𐎵 19	𐎶𐎶𐎶𐎶𐎶𐎶𐎶 29	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎵 39	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎶 49	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎵 59
𐎶𐎶𐎶𐎶𐎶𐎶𐎵 10	𐎶𐎶𐎶𐎶𐎶𐎶𐎶 20	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎵 30	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎶 40	𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎶𐎵 50	

They don't have a symbol for 0

Trivia: The Babylonians, famous for their astronomical observations, used a base-60 number system!

- 60 seconds in a minute
- 60 minutes in an hour
- 360 degrees in a circle

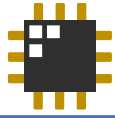


10

Image credit: https://en.wikipedia.org/wiki/Babylonian_cuneiform_numerals

Also, did you know that the Ancient Babylonians used a base-60 number system? They would have 60 symbols to represent each number. Note that they didn't really have a symbol for 0, so the drawings only show 59 symbols.

This base-60 number system is mainly why we have 60 seconds in a minute, 60 minutes in an hour, and 360 degrees in a circle!



Other bases

- Segue: How about base-64?

<https://www.youtube.com/watch?v=dQw4w9WgXcQ>

Trivia: Youtube **video IDs** are encoded in base-64!

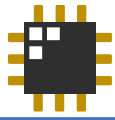


Base 64 = 64 symbols

- a, b, c, ..., z = **26** symbols
 - A, B, C, ..., Z = **26** symbols
 - 0, 1, 2, ..., 9 = **10** symbols
 - - and _ = **2** symbols
- $26+26+10+2 = 64$

11

Also, did you know that the Ancient Babylonians used a base-60 number system? They would have 60 symbols to represent each number. Note that they didn't really have a symbol for 0, so the drawings only show 59 symbols. This base-60 number system is mainly why we have 60 seconds in a minute, 60 minutes in an hour, and 360 degrees in a circle!



Binary number conversion

- This demonstrates conversion from binary to decimal
 - For every bit position **going to the left, we increase the power of 2**, then just **multiply the actual bit** to that power of 2

$$\begin{aligned} & \mathbf{1110}_2 \\ & (\mathbf{1} \times 2^3) + (\mathbf{1} \times 2^2) + (\mathbf{1} \times 2^1) + (\mathbf{0} \times 2^0) \\ & \mathbf{1} \times 8 + \mathbf{1} \times 4 + \mathbf{1} \times 2 + \mathbf{0} \times 1 \\ & = \mathbf{14}_{10} \end{aligned}$$

12

Let's delve into the conversion from binary to decimal using the binary number 1110 as an example.

From the math shown earlier, which is also replicated here, for every bit position **going to the left, we increase the power of 2**, then just **multiply the actual bit** to that power of 2, add them up then we have our decimal equivalent.

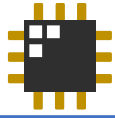
In the binary system, each digit (or bit) represents a power of 2. Starting from the rightmost bit (also known as the least significant bit), we have 2^0 , then moving to the left, we increase the power by 1 for each bit position.

So, for the binary number 1110:

- The rightmost bit is 0, and it's in the position of 2^0 . So, we calculate $0 * 2^0 = 0$.
- The next bit to the left is 1, and it's in the position of 2^1 . So, we calculate $1 * 2^1 = 2$.
- The next bit to the left is 1, and it's in the position of 2^2 . So, we calculate $1 * 2^2 = 4$.
- The leftmost bit (also known as the most significant bit) is 1, and it's in the position of 2^3 . So, we calculate $1 * 2^3 = 8$.

Finally, we add up these values: $8 + 4 + 2 + 0 = 14$. So, 1110 in binary is

equivalent to 14 in decimal. This is how we convert from binary to decimal.



Binary number conversion

- Practice, what is the decimal equivalent of this?

10001111_2

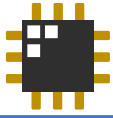
13

Let's practice, what is the decimal equivalent of this binary number?

$$1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1 \times 128 + 0 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1$$

$$128 + 8 + 4 + 2 + 1 = 143$$



Binary number conversion

- How to do the reverse? Brute-force way: Successive division

- Divide the number by the base (in this case 2). Get the quotient.
- Continuously divide the quotient by the base until it becomes 0.
- Always take note of the remainder each time the quotient is divided.
- Arrange the order of remainders. The last remainder is at the left-most position.

$$14 / 2 = 7 \text{ remainder } 0$$

$$7 / 2 = 3 \text{ remainder } 1$$

$$3 / 2 = 1 \text{ remainder } 1$$

$$1 / 2 = 0 \text{ remainder } 1$$

↑
Read in this direction

1110₂

14

Now, let's try to do the reverse process: converting a decimal number into a binary number.

Divide the number by the base (in this case 2). Get the quotient.

Continuously divide the quotient by the base until it becomes 0.

Always take note of the remainder each time the quotient is divided.

Arrange the order of remainders. The last remainder is at the left-most position.

Let's convert the decimal number 14 to binary using successive division:

1. Divide 14 by 2, which equals 7 with a remainder of 0. So, the first digit of the binary number (from right to left) is 0.

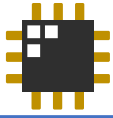
2. Divide the quotient 7 by 2, which equals 3 with a remainder of 1. So, the next digit is 1.

3. Divide the quotient 3 by 2, which equals 1 with a remainder of 1. So, the next digit is 1.

4. Divide the quotient 1 by 2, which equals 0 with a remainder of 1. So, the next digit is 1.

Now that the quotient is 0, we stop the process. Reading the remainders from bottom to top (or from right to left), we get the binary number 1110. So, 14 in

decimal is equivalent to 1110 in binary. This is how we convert from decimal to binary using successive division.



Binary number conversion

- How to do the reverse? Faster(?) way

- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$(_x2^3) + (_x2^2) + (_x2^1) + (_x2^0) = 14$$

15

While successive division works, it can be tedious, especially for large numbers. A relatively faster way is to directly write out the binary string slowly starting from the left-most bit then working your way to the right.

This relies on you memorizing the powers of 2, or at least be familiar with it, and using subtraction. Here's how you can do it:

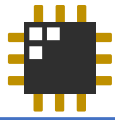
1. Memorize the powers of 2: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, and so on.

2. Start with the highest power of 2 that is less than or equal to the number you're converting. For 14, this would be 8 (2^3).

3. Subtract this power of 2 from your number. For 14, you would subtract 8 to get 6.

4. Move to the next lower power of 2 (which is 4 or 2^2 in this case) and see if you can subtract it from the remainder without getting a negative number. If you can, subtract it and put a 1 in that bit position. If you can't, put a 0 in that bit position. For 6, you can subtract 4, so you put a 1 in the 2^2 position.

5. Repeat this process with the new remainder and the next lower power of 2. Continue until you reach 2^0 .



Binary number conversion

- How to do the reverse? Faster(?) way

- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

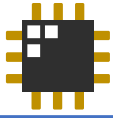
$$\begin{array}{ccccccc} (1 \times 2^3) & + & (_ \times 2^2) & + & (_ \times 2^1) & + & (_ \times 2^0) = 14 \\ 8 & + & ? & + & ? & + & ? = 14 \end{array}$$

16

So, for 14, the process would look like this:

- $14 - 8 (2^3) = 6$, so the bit at position 2^3 is 1.
- $6 - 4 (2^2) = 2$, so the bit at position 2^2 is 1.
- $2 - 2 (2^1) = 0$, so the bit at position 2^1 is 1.
- Finally, there's nothing to subtract from 0, so the bit at position 2^0 is 0.

So, 14 in decimal is 1110 in binary. This method can be faster because it eliminates the need for division and focuses on subtraction and simple binary decisions (can I subtract this power of 2 or not?). However, it does require you to have the powers of 2 memorized, at least up to the range of numbers you're working with.



Binary number conversion

- How to do the reverse? Faster(?) way

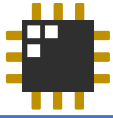
- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$\begin{array}{ccccccc} (1 \times 2^3) & + & (1 \times 2^2) & + & (_ \times 2^1) & + & (_ \times 2^0) = 14 \\ 8 & + & 4 & + & ? & + & ? = 14 \end{array}$$

17

- $6 - 4 (2^2) = 2$, so the bit at position 2^1 is 1.



Binary number conversion

- How to do the reverse? Faster(?) way

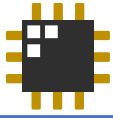
- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$\begin{array}{ccccccc} (1 \times 2^3) & + & (1 \times 2^2) & + & (1 \times 2^1) & + & (_ \times 2^0) & = & 14 \\ 8 & + & 4 & + & 2 & + & ? & = & 14 \end{array}$$

18

- $14 - 8 = 6$, so the bit at position 2^3 is 1.



Binary number conversion

- How to do the reverse? Faster(?) way

- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

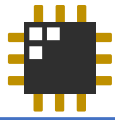
We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$\begin{array}{rcccccc} (1 \times 2^3) & + & (1 \times 2^2) & + & (1 \times 2^1) & + & (0 \times 2^0) & = & 14 \\ 8 & + & 4 & + & 2 & + & 0 & = & 14 \end{array}$$

19

- Finally, there's nothing to subtract from 0, so the bit at position 2^0 is 0.

So, 14 in decimal is 1110 in binary. This method can be faster because it eliminates the need for division and focuses on subtraction and simple binary decisions (can I subtract this power of 2 or not?). However, it does require you to have the powers of 2 memorized, at least up to the range of numbers you're working with.



Binary number conversion

- Practice: Write 43 in binary. You can use either method.

43₁₀

20

Let's have another practice. Try converting 43 in binary.

Using the division method:

$$43/2 = 21 \text{ remainder } 1$$

$$21/2 = 10 \text{ remainder } 1$$

$$10/2 = 5 \text{ remainder } 0$$

$$5/2 = 2 \text{ remainder } 1$$

$$2/2 = 1 \text{ remainder } 0$$

$$1/2 = 0 \text{ remainder } 1$$

Arrange the remainder from the bottom going up. $101011 = 43$.

The process looks short, but I personally find it rather tedious. Now, let's try to use the subtraction process described earlier to do the conversion.

Largest power of 2 that can be subtracted from 43 is 32. 32 is 2^5 .

Put 1 on the 2^5 position. We start from there.

$$43 - 32 = 11 \text{ (the number we represent next)}$$

2^4 position has weight of 16, don't subtract from 11 (because $11-16$ is negative). Put 0 on the 2^4 position.

2^3 position has weight of 8, we can subtract 8 from 11. Put 1 on the 2^3 position.

$11 - 8 = 3$ (the number we represent next)

2^2 position has weight of 4, don't subtract from 3 (because $3 - 4$ is negative). Put 0 on the 2^2 position.

2^1 position has weight of 2, we can subtract 2 from 3. Put 1 on the 2^1 position.

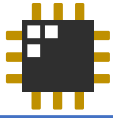
$3 - 2 = 1$ (the number we represent next)

2^0 position has weight of 1, we can subtract 1 from 1, so put 1 on the 2^0 position.

At this point, we have already reached the bit position 2^0 , so we end here.

$43 = 101011$

The process looks lengthy because I'm explaining the process along the way, but once you get the hang of it, I guarantee that it can be a faster method for conversion!

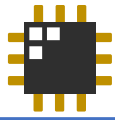


Converting numbers to other bases

- We can represent any number in any base n , that has n digits with positional powers of n
- Use the same procedures shown in the previous slides but change the base number. For example, to convert 123_5 to decimal:

$$\begin{aligned} &123_5 \\ &(1 \times 5^2) + (2 \times 5^1) + (3 \times 5^0) \\ &25 + 2 \times 5 + 3 \times 1 \\ &= 38_{10} \end{aligned}$$

21



Number conversion

- Practice: Convert 310_5 to base 7
 - Tip: You can use base-10 as an intermediate base during conversion

310_5

22

310 in base-5 is:

$$3 \times 5^2 + 1 \times 5^1 + 0 \times 5^0 = 3 \times 25 + 1 \times 5 + 0 \times 1 = 75 + 5 = 80 \text{ in base 10}$$

=====

Now, convert 80 in base 10 to base 7. We can use the successive division approach:

Divide 80 by 7:

$$80 \div 7 = 11 \text{ with a remainder of } 3. \text{ (Quotient is 11, remainder is 3)}$$

Next, divide the quotient (11) by 7:

$$11 \div 7 = 1 \text{ with a remainder of } 4. \text{ (Quotient is 1, remainder is 4)}$$

Next, divide the quotient (1) by 7:

$$1 \div 7 = 0 \text{ with a remainder of } 1. \text{ (Quotient is 0, remainder is 1)}$$

Combine the remainders:

The remainders, read from bottom to top, give you the base 7 number.

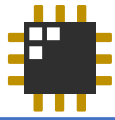
So, 80 in base 10 is equal to 143 in base 7.

=====

We can also approach this using the alternative subtraction technique:

1. Identify the largest power of 7 less than 80, which is 7^2 (49).
2. Determine the coefficient for 7^2 by dividing 80 by 49:
 1. $80 \div 49 = 1$ (coefficient) with a remainder of 31.
3. Next, look at the remainder (31) and identify the largest power of 7 less than 31, which is 7^1 (7).
4. Determine the coefficient for 7^1 by dividing 31 by 7:
 1. $31 \div 7 = 4$ (coefficient) with a remainder of 3.
5. Now, look at the remainder (3), which is less than 7 and doesn't require further division.
6. Combine the coefficients from the highest power down to zero:
 1. The coefficient for 7^2 is 1.
 2. The coefficient for 7^1 is 4.
 3. The coefficient for 7^0 is 3.

So, we get 143 in base 7, confirming our previous result.



Hexadecimal numbers

- Another common base is base 16 (called hexadecimal or hex)

4-bit binary	Hex value	Decimal value
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7

4-bit binary	Hex value	Decimal value
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

- In hex, there are 16 digits (0 to F).
- We usually write it with 0x prefix (e.g. 0xA, 0xF, 0xAF)

23

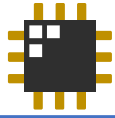
Another common base that comes up in computing is base 16, also called hexadecimal number or hex for short.

In hex, we have 16 digits, because we are in base 16. The 16 digits are represented by symbols going from 0 to 9, then letters A to F. 0 to F is actually 16 items, those will be our digits.

Here's the same table earlier for a 4-bit binary string and the equivalent decimal value. Then, we added the corresponding hex value as well. Since this table is 16 items, everything fits in a single digit of hex. You can see their equivalence here. The digits are similar to decimal from 0 to 9, but once we exceed 9, hex goes to A, then B, then C, and so on until F.

For hex, we usually write it with a 0x prefix (examples: 0xA, 0xF, 0xAF)

This set of 4-bit binary, the equivalent hex digit, and the corresponding decimal value comes up a lot in the world of computing.



Binary to hex conversion

- Practice: Write 1010 1101 1111 0001 in hex

1010 1101 1110 0001₂

24

Let's have some practice. Try writing 1010 1101 1111 0001 from binary to hex.
Straight conversion from the table (or if you memorized it)

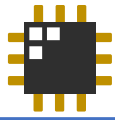
1010 = A

1101 = D

1110 = E

0001 = 1

0xADE1



Arbitrary base conversion

- Practice: What is 0xAF in base-6?

$$0xAF = AF_{16}$$

25

Here's another one, converting a hex number to base-6. We first convert to decimal.
What is the decimal equivalent of 0xAF?
There are several ways to do this:

Straight from hex to decimal. Take note how every digit position in hex is a power of 16.

$$A \times 16^1 = 10 \times 16 = 160$$

$$F \times 16^0 = 15 \times 1 = 15$$

$$160 + 15 = \mathbf{175}$$

Can convert to binary, then binary to decimal.

AF = 1010 1111 (using the table or if you memorized it already)

Now that we are in the binary domain, we can start doing the math from binary to decimal. Every bit position in binary is a power of 2.

$$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1 \times 128 + 0 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1$$

$$128 + 32 + 8 + 4 + 2 + 1 = \mathbf{175}$$

=====

Now, we convert 175 to base-6

Successive division method:

1. Divide 175 by 6:

$175 \div 6 = 29$ with a remainder of 1. (Quotient is 29, remainder is 1)

2. Next, divide the quotient (29) by 6:

$29 \div 6 = 4$ with a remainder of 5. (Quotient is 4, remainder is 5)

3. Next, divide the quotient (4) by 6:

$4 \div 6 = 0$ with a remainder of 4. (Quotient is 0, remainder is 4)

4. Combine the remainders:

The remainders, read from bottom to top, give you the base 6 number.

So, 175 in base 10 is equal to **451 in base 6**.

Repeated subtraction method:

1. Identify the largest power of 6 less than 175, which is 6^2 (36).

2. Determine the coefficient for 6^2 by dividing 175 by 36:

1. $175 \div 36 = 4$ (coefficient) with a remainder of 31.

3. Next, look at the remainder (31) and identify the largest power of 6 less than 31, which is 6^1 (6).

4. Determine the coefficient for 6^1 by dividing 31 by 6:

1. $31 \div 6 = 5$ (coefficient) with a remainder of 1.

5. Now, look at the remainder (1), which is less than 6 and doesn't require further division.

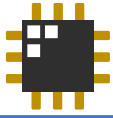
6. Combine the coefficients from the highest power down to zero:

1. The coefficient for 6^2 is 4.

2. The coefficient for 6^1 is 5.

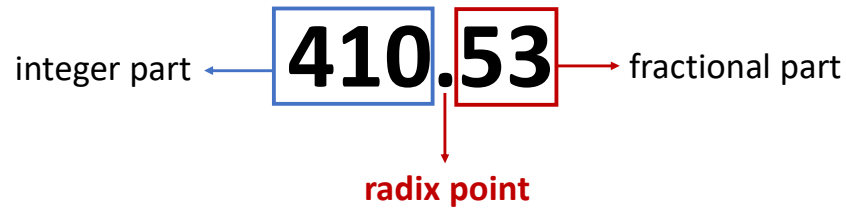
3. The coefficient for 6^0 is 1.

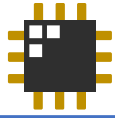
So, we get **451 in base 6**, confirming our previous result.



What about fractions?

- In positional number systems, the **radix point** separates the integer part of a number from its fractional part





What about fractions?

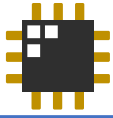
- For decimal numbers, the digit on the left is 10 times higher than the digit on the right.

410.53

$$(4 \times 10^2) + (1 \times 10^1) + (0 \times 10^0) + (5 \times 10^{-1}) + (3 \times 10^{-2})$$

This part of the conversion has
already been covered

Once we reach the fractional
part, the exponents go negative



Binary to decimal conversion with fractions

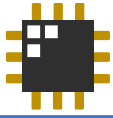
- Use negative powers or multiplicative inverses of the base for place values after the radix point, i.e. fractional part

1011.101₂

1	$\times 2^{-3} = 1(1/2^3) = 0.125$
0	$\times 2^{-2} = 0(1/2^2) = 0$
1	$\times 2^{-1} = 1(1/2^1) = 0.5$
1	$\times 2^0 = 1(1) = 1$
1	$\times 2^1 = 1(2) = 2$
0	$\times 2^2 = 0(4) = 0$
1	$\times 2^3 = 1(8) = 8$

$$0.125 + 0 + 0.5 + 1 + 2 + 0 + 8 = \mathbf{11.625_{10}}$$

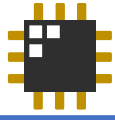
28



Binary to decimal conversion with fractions

- Convert the integer part and the decimal part separately
 - For the integer part: use the same procedure previously shown
 - For the fractional part:
 1. Multiply the fractional part by the base. Note the integer part of the product; use the fractional part of the product for the next step
 2. Repeat step 1 for as many times as the desired number of places after the radix point, taking note of the integer part for each obtained product
 3. Arrange the obtained integers such that the first integer you obtained is the first number to the right of the radix point

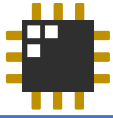
An example will make this clearer 😊



Binary to decimal conversion with fractions

- Example: Convert 0.7_{10} to binary up to 5 bits after the radix point

0.7_{10}



Binary to decimal conversion with fractions

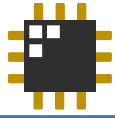
- Example: Convert 0.7_{10} to binary up to 5 bits after the radix point

$$(0.7) \times 2 = 1.4$$

Multiply the number by the base (in this case 2)

Take note of the integer part

Use the fraction part for the next step



Decimal to binary conversion with fractions

- Example: Convert 0.7_{10} to binary up to 5 bits after the radix point

Continue multiplying the fraction part of the product by 2

$$\begin{array}{l} (0.7) \times 2 = 1.4 \\ (0.4) \times 2 = 0.8 \\ (0.8) \times 2 = 1.6 \\ (0.6) \times 2 = 1.2 \\ (0.4) \times 2 = 0.4 \end{array}$$

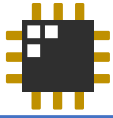
Take note of the integer part

Read in this direction

$$0.7_{10} \approx 0.10110_2$$

Note that this is just an approximation. We increasing the number of bits to get closer to the actual value

32



Decimal to binary conversion with fractions

$$14 / 2 = 7 \text{ remainder } 0$$

$$7 / 2 = 3 \text{ remainder } 1$$

$$3 / 2 = 1 \text{ remainder } 1$$

$$1 / 2 = 0 \text{ remainder } 1$$

Read in this direction

When converting decimal
integers to binary

$$(0.7) \times 2 = 1.4$$

$$(0.4) \times 2 = 0.8$$

$$(0.8) \times 2 = 1.6$$

$$(0.6) \times 2 = 1.2$$

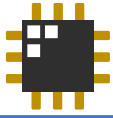
$$(0.4) \times 2 = 0.4$$

Read in this direction

When converting decimal
fractions to binary

Is there a more *intuitive* way?

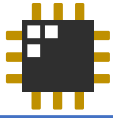
YES!



Binary to decimal conversion with fractions

- You can also technically do the *faster* method even with fractions!
 - Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.
 - Note that powers of 2 to the right of the radix point goes negative!

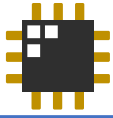
$$(_x2^{-1}) + (_x2^{-2}) + (_x2^{-3}) + (_x2^{-4}) + (_x2^{-5}) = 0.7$$



Binary to decimal conversion with fractions

- You can also technically do the *faster* method even with fractions!
 - Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.
 - Note that powers of 2 to the right of the radix point goes negative!

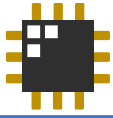
$$\begin{array}{ccccccccc} (1 \times 2^{-1}) & + & (_ \times 2^{-2}) & + & (_ \times 2^{-3}) & + & (_ \times 2^{-4}) & + & (_ \times 2^{-5}) & = & 0.7 \\ 0.5 & + & ? & + & ? & + & ? & + & ? & = & 0.7 \end{array}$$



Binary to decimal conversion with fractions

- You can also technically do the *faster* method even with fractions!
 - Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.
 - Note that powers of 2 to the right of the radix point goes negative!

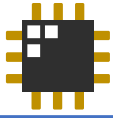
$$\begin{array}{ccccccccc} (1 \times 2^{-1}) & + & (0 \times 2^{-2}) & + & (_ \times 2^{-3}) & + & (_ \times 2^{-4}) & + & (_ \times 2^{-5}) & = & 0.7 \\ 0.5 & + & 0 & + & ? & + & ? & + & ? & = & 0.7 \end{array}$$



Binary to decimal conversion with fractions

- You can also technically do the *faster* method even with fractions!
 - Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.
 - Note that powers of 2 to the right of the radix point goes negative!

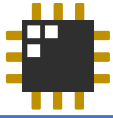
$$\begin{array}{ccccccccc} (1 \times 2^{-1}) & + & (0 \times 2^{-2}) & + & (1 \times 2^{-3}) & + & (_ \times 2^{-4}) & + & (_ \times 2^{-5}) & = & 0.7 \\ 0.5 & + & 0 & + & 0.125 & + & ? & + & ? & = & 0.7 \end{array}$$



Binary to decimal conversion with fractions

- You can also technically do the *faster* method even with fractions!
 - Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.
 - Note that powers of 2 to the right of the radix point goes negative!

$$\begin{array}{rccccccccc} (1 \times 2^{-1}) & + & (0 \times 2^{-2}) & + & (1 \times 2^{-3}) & + & (1 \times 2^{-4}) & + & (_ \times 2^{-5}) & = & 0.7 \\ 0.5 & + & 0 & + & 0.125 & + & 0.0625 & + & ? & = & 0.7 \end{array}$$



Binary to decimal conversion with fractions

- You can also technically do the *faster* method even with fractions!
 - Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.
 - Note that powers of 2 to the right of the radix point goes negative!

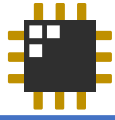
$$(1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) + (1 \times 2^{-4}) + (0 \times 2^{-5}) = 0.7$$

$$0.5 + 0 + 0.125 + 0.0625 + 0 \approx 0.7$$

only approximately

You can be more accurate
as you add more bits!

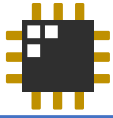
39



Binary number conversion

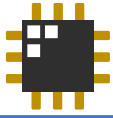
- Practice, write 143.85 from decimal to binary

143.85₁₀



Number conversion with fractions

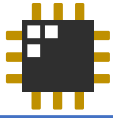
- Positional number systems have an ordered set of digits whose values depend on their position
 - Decimal, binary, and hexadecimal are examples of positional number systems
- **Any *base-n* number can be converted to decimal and vice-versa.**
Procedures for converting between bases involve computing the equivalent values of each digit
 - Positional powers increase as you go to the left-hand side digits
 - Positional powers decrease as you go to the right-hand side digits



Representing numbers in computer systems

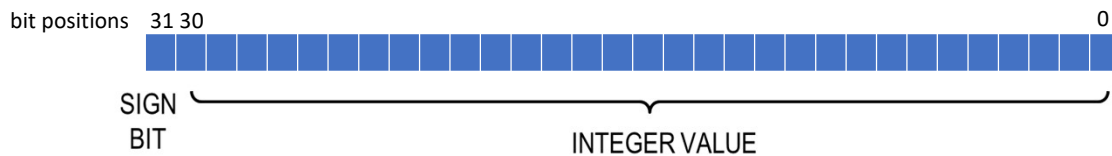
- We observed that converting decimal fractions to binary introduces some loss of accuracy
- Something to consider: Numbers are continuous and infinite, but computers are discrete and finite

How do we effectively represent very large and very small numbers using a limited number of bits?

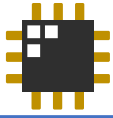


Range of binary numbers: Integers

- With n bits of storage, we can represent 2^n different values
- Example:
 - We have 32 bits and we need to represent both positive and negative numbers (use the leftmost bit as a sign bit)
 - We can represent $2^{32} = 4,294,967,296$ numbers
 - Ranging from $-2,147,483,648$ to $2,147,483,647$
 - **But these are only integers!**



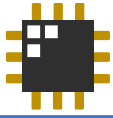
43



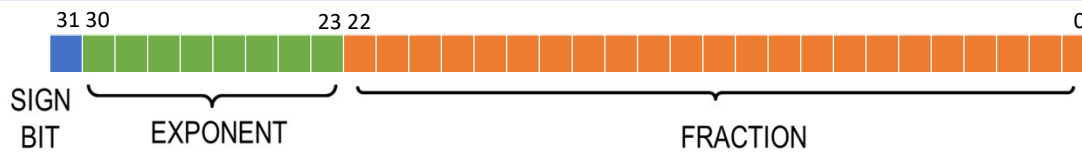
Floating-point representation

- **Floating-point notation:** $F \times r^E$
 - F – fraction
 - E – exponent
 - r – radix or base

Just like scientific/engineering notation!
- The fraction can be **normalized** such that there is a single non-zero digit before the radix point
 - Example: 1010.1101_2 can be normalized and expressed in floating-point as $1.0101101_2 \times 2^3$



IEEE-754 32-bit floating-point numbers

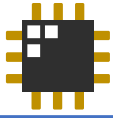


$$F \times r^E$$

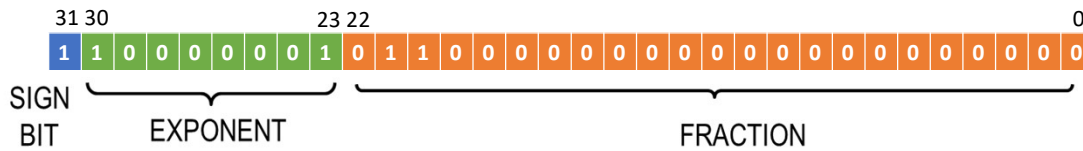
- Most significant bit (bit 31) is the sign bit (0 if positive, 1 if negative)
- The succeeding 8 bits (bit 30 down to bit 23) represent E , the exponent
- Remaining bits represent (bit 22 down to bit 0) represent F , the fraction
- $r = 2$ because we are using binary

Computers use the IEEE-754 standard to represent floats!

45

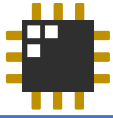


Example: Convert the number to decimal

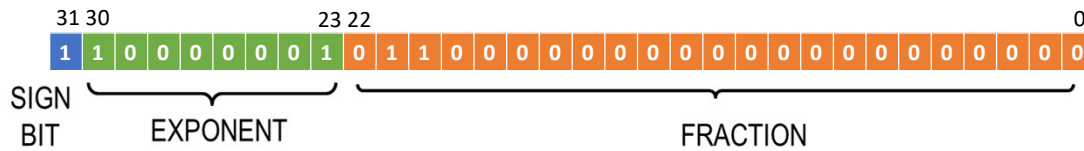


- $S = 1$ (negative number)
- $E = 1000\ 0001$
 - We compute the exponent as $E - 127$ (defined by IEEE-754 standard). This allows us to represent positive and negative exponents
 - $1000\ 0001_2 = 129_{10}$
 - Thus, the actual exponent (in decimal) is $129 - 127 = 2$
- $F = 011\ 0000\ 0000\ 0000\ 0000\ 0000$
 - Normalized form implies a '1' before the radix point
 - $1.F = 1.011000000000000000000000_2$
 - In decimal, this is $1 + 1(1/2^2) + 1(1/2^3) = 1 + 0.25 + 0.125 = \mathbf{1.375}$

46



Example: Convert the number to decimal

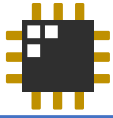


$$F \times r^E$$

- Number is negative
- Exponent = 2
- $1.F = 1.375_{10}$
- Radix = 2
- Therefore, the number being represented is -1.375×2^2 or -5.5

47





IEEE-754 number conversion

```
import struct
```

```
def float_to_ieee_754_binary(value):  
    # Convert float to binary using struct  
    packed_value = struct.pack('!f', value)  
    # Convert packed value to integer representation of binary  
    integer_value = struct.unpack('!I', packed_value)[0]  
    # Format integer value as binary string  
    binary_string = f'{integer_value:032b}'  
    return binary_string
```

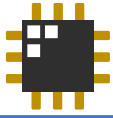
```
number = -5.5  
binary_representation = float_to_ieee_754_binary(number)  
print(f"The IEEE-754 binary representation of {number} is: {binary_representation}")
```

Try this out on your own!

This Python code transforms a floating point number into the corresponding binary representation as it is stored in the computer memory!

Computers use the IEEE-754 standard to represent floats!

49

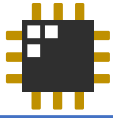


Additional notes on floating-point representation

- We need a de-normalized form to represent 0
 - We indicate the de-normalized form when $E=0$, actual exponent is always -126
 - Instead of an implied '1' before the radix point, we assume '0' before the radix point in de-normalized form
- Double-precision uses 64 bits, F takes up 53 bits
- Floating-point numbers are not faithful representations of mathematical real numbers (more on this here: [What you never wanted to know about floating point but will be forced to find out](https://www.volkerschatz.com/science/float.html))
 - Finite number of floats
 - Only fractions with power-of-two denominators are representable accurately
 - Rounding off or truncating fractional bits causes a loss of accuracy

50

<https://www.volkerschatz.com/science/float.html>



Optional: Try this out

(Example taken from Numerical Methods, by Greenbaum and Chartier)

- By hand, write down the output of the following python code for 10 iterations if you start with $x = 0.1$

```
def f(x):  
    if x <= 0.5:  
        return 2*x  
    if x > 0.5:  
        return 2*x - 1
```

- Run the function for 80 times, starting with $x=0.1$ and see what happens. What went wrong?

```
x = 0.1  
for i in range(80):  
    print(x)  
    x = f(x)
```

51

Try this out in python and see what happens. Based on what you learned from the binary number system and floating point representation, can you explain the result?

```
def f(x):  
    if x <= 0.5:  
        return 2*x  
    if x > 0.5:  
        return 2*x - 1
```

```
x = 0.1  
for i in range(80):  
    print(x)  
    x = f(x)
```